
MINISPAMFILTER: MULTICLASS SMS CLASSIFICATION USING NAIVE BAYES IN C++ *

Rohit Reddy
Dept. of Data Science & AI
IIIT Bangalore
rohit.reddy@iiitb.ac.in

Dhruv Reddy
Dept. of Data Science & AI
IIIT Bangalore
Reddy.Sambavaram@iiitb.ac.in

Nitheesh Kumar
Dept. of Electronics & Communication
IIIT Bangalore
Nitheesh.Kumar@iiitb.ac.in

Jithindra Reddy
Dept. of Data Science & AI
IIIT Bangalore
Jithindra.Reddy@iiitb.ac.in

ABSTRACT

We present *MiniSpamFilter*, a fully object-oriented C++ implementation of a multinomial Naive Bayes classifier for three-way SMS classification (Ham, Spam, Scam/Smishing). The system features a clean class hierarchy with interchangeable tokenisers, Laplace (add-one) smoothing, log-space probability computation, stratified 80/20 evaluation, and an interactive prediction interface. On a balanced public SMS dataset of 10 191 messages the model achieves an accuracy of **94.55 %** and a macro-averaged F1-score of **0.9456**. We describe the mathematical foundations, software architecture, experimental protocol and a detailed error analysis.

Keywords Naive Bayes · SMS classification · Spam detection · Smishing · Laplace smoothing · C++ · Object-oriented design

1 Introduction

Short Message Service (SMS) remains one of the most ubiquitous communication channels. Consequently it is also a primary vector for unsolicited commercial messages (spam) and, more dangerously, for phishing attacks delivered via SMS—commonly called *smishing*. Distinguishing legitimate messages (*Ham*) from ordinary spam and from sophisticated social-engineering scams is therefore a practically important multiclass text-classification problem.

Naive Bayes remains an attractive baseline for this task: it is fast to train, requires only modest memory, is robust to high-dimensional sparse feature spaces, and admits a transparent probabilistic interpretation. In this work we engineer a production-quality C++ realisation of multinomial Naive Bayes that emphasises:

- clean object-oriented design (inheritance for tokenisers, clear separation of concerns);
- numerically stable log-space arithmetic together with Laplace smoothing;
- a complete evaluation suite (accuracy, per-class precision/recall/F1, confusion matrix);
- an interactive command-line interface for live classification.

The remainder of the paper is organised as follows. Section 2 recalls the multinomial Naive Bayes model and Laplace smoothing. Section 3 presents the software architecture. Section 4 describes the data and experimental protocol. Section 5 reports quantitative results and an error analysis. Section 7 concludes.

* *Citation:* Rohit Reddy, Dhruv Reddy, Nitheesh Kumar, Jithendra Reddy. *MiniSpamFilter: Multiclass SMS Classification using Naive Bayes in C++*. IIIT Bangalore, 2026.

2 Background and Mathematical Formulation

2.1 Multinomial Naive Bayes

Let $\mathcal{C} = \{c_1, \dots, c_K\}$ be the set of class labels (here $K = 3$: Ham, Spam, Scam). A message d is represented as a bag-of-words vector $\mathbf{x} = (x_1, \dots, x_{|V|})$, where:

- V is the vocabulary (all distinct tokens seen in training),
- x_t is the number of times token t occurs in d .

By Bayes' theorem the class posterior satisfies

$$P(c | d) \propto P(c) P(d | c), \quad (1)$$

where $P(c)$ is the *prior* probability of class c and $P(d | c)$ is the *likelihood* of the document given that class.

Under the multinomial event model and the Naive Bayes assumption (tokens are conditionally independent given the class),

$$P(d | c) = \frac{(\sum_t x_t)!}{\prod_t x_t!} \prod_{t=1}^{|V|} P(t | c)^{x_t}. \quad (2)$$

The factorial prefactor does not depend on c and can be dropped when comparing classes. Taking the logarithm (which does not change the $\arg \max$) yields the decision rule used in the implementation:

$$\hat{c}(d) = \arg \max_{c \in \mathcal{C}} \left(\log P(c) + \sum_{t \in d} x_t \log P(t | c) \right). \quad (3)$$

Here $\log P(c)$ is the log-prior and $\log P(t | c)$ is the *log-likelihood* of token t under class c .

2.2 Parameter Estimation with Laplace Smoothing

Maximum-likelihood estimates of $P(t | c)$ can be zero when token t never appears with class c in the training set, which makes the whole product in Eq. (2) zero. We therefore apply Laplace (add-one) smoothing:

$$P(t | c) = \frac{N_{t,c} + 1}{N_c + |V|}, \quad (4)$$

where:

- $N_{t,c}$ = number of times token t occurs in training documents of class c ,
- $N_c = \sum_t N_{t,c}$ = total number of tokens in class c ,
- $|V|$ = vocabulary size.

The corresponding **log-likelihood** used in Eq. (3) is

$$\log P(t | c) = \log(N_{t,c} + 1) - \log(N_c + |V|). \quad (5)$$

The class prior is the relative frequency of documents:

$$P(c) = \frac{N_c^{\text{doc}}}{N^{\text{doc}}}, \quad (6)$$

where N_c^{doc} is the number of training documents labelled c and N^{doc} is the total number of training documents. In log-space,

$$\log P(c) = \log N_c^{\text{doc}} - \log N^{\text{doc}}. \quad (7)$$

All scores in the C++ implementation are computed with Eqs. (3)–(5) so that products of many small probabilities never underflow to zero.



Figure 1: UML class diagram of MiniSpamFilter.

3 System Design

3.1 Class Hierarchy

Figure 1 shows the UML class diagram of the system.

The principal abstractions are:

- **Document** – holds the raw text, its gold label and a word-count map produced by a tokeniser.

- `ITokeniser` (interface) together with two concrete implementations:
 - `SimpleTokeniser` – lower-cases and splits on non-alphanumeric characters;
 - `StopWordTokeniser` – extends the simple tokeniser by removing a fixed English stop-word list.
- `Vocabulary` – a set of unique tokens observed during training.
- `NaiveBayesClassifier` – implements training (Eq. 4) and prediction (Eq. 3).
- `Evaluator` – computes accuracy, per-class precision/recall/F1 and the confusion matrix.

The design follows the Open-Closed Principle: new tokenisers can be added without modifying the classifier.

3.2 Training and Prediction Pipeline

1. Load CSV files; each row yields a Document.
2. Tokenise every document (stop-word tokeniser is the default).
3. Call `NaiveBayesClassifier::train` which accumulates class-document counts, class-token counts and builds the vocabulary.
4. For evaluation, predict the label of every test document and feed the resulting vectors to the `Evaluator`.
5. Optionally enter an interactive loop that classifies free-form SMS messages.

4 Dataset and Experimental Setup

We use a publicly available balanced SMS collection containing 10 191 messages equally distributed among the three classes (Ham, Spam, Smishing). Smishing messages are relabelled Scam for clarity. A stratified 80/20 split (random seed 42) produces:

Table 1: Class distribution after the stratified split.

Split	Ham	Spam	Scam	Total
Train	2 718	2 718	2 718	8 154
Test	679	679	679	2 037

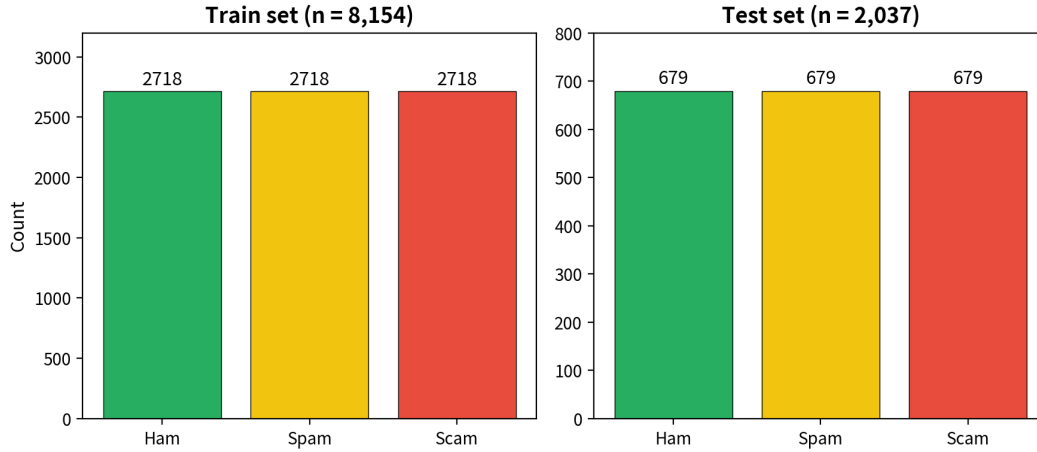


Figure 2: Class distribution in the train and test partitions.

All experiments were performed with the stop-word tokeniser and Laplace smoothing ($\alpha = 1$).

5 Results and Discussion

5.1 Overall Performance

On the held-out test set the classifier attains:

Accuracy = 94.55 % (1 926 / 2 037 correct)

Macro-averaged metrics are summarised in Table 2.

Table 2: Per-class and macro-averaged evaluation metrics.

Class	Precision	Recall	F1-score
Ham	0.9910	0.9779	0.9844
Spam	0.9116	0.9411	0.9261
Scam	0.9354	0.9175	0.9264
Macro	0.9460	0.9455	0.9456

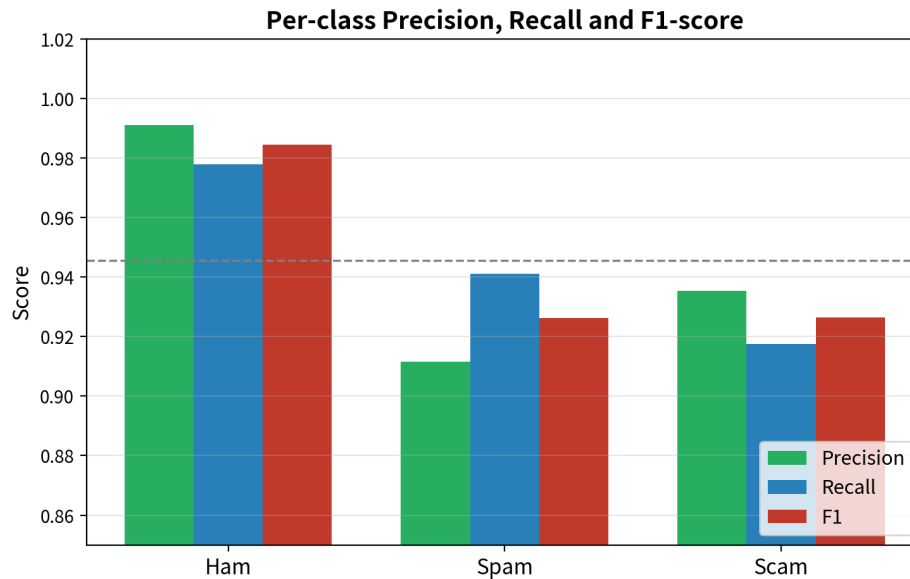


Figure 3: Per-class Precision, Recall and F1-score.

5.2 Confusion Matrix

Table 3: Confusion matrix (rows = true label, columns = predicted label).

	Ham	Spam	Scam
Ham	664	10	5
Spam	2	639	38
Scam	4	52	623

5.3 Error Analysis

The dominant error modes are:

- **Spam ↔ Scam confusion** (38 + 52 = 90 messages). Both classes share promotional language, URLs and urgency cues; the distinction is often semantic rather than lexical.
- **Ham misclassified as Spam/Scam** (15 messages). Legitimate messages that contain numbers, short codes or informal slang occasionally trigger the spam/scam priors.

Ham is recognised almost perfectly (F1 = 0.984), confirming that ordinary conversational language is well separated from the other two classes.

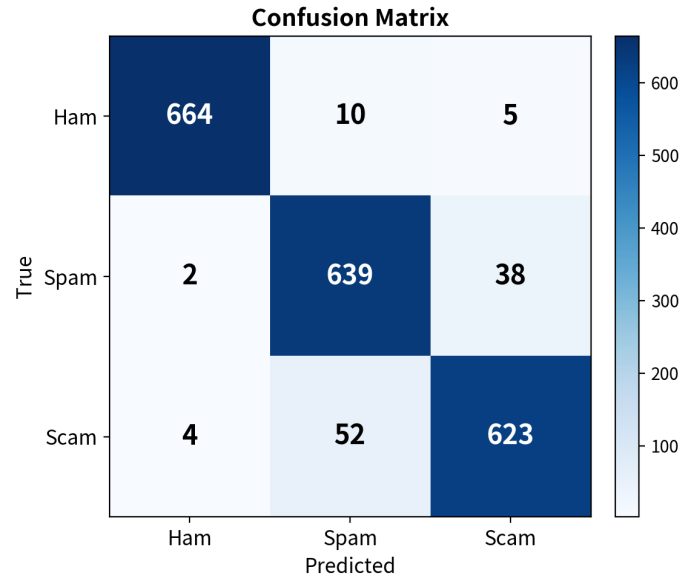


Figure 4: Heat-map visualisation of the confusion matrix.

6 Implementation Notes

The project is built with CMake (C++11) and consists of approximately 1 200 lines of code. A Windows batch script (`run.bat`) and a POSIX build path are both provided. Key design decisions include:

- All probability arithmetic is performed with `std::log` to guarantee numerical stability.
- The vocabulary and count tables are stored in `std::map` for simplicity and deterministic ordering.
- CSV parsing correctly handles quoted fields and UTF-8 BOM.

7 Conclusion and Future Work

We have presented a carefully engineered multinomial Naive Bayes SMS classifier that reaches 94.55 % accuracy on a balanced three-class benchmark. The object-oriented design, Laplace smoothing and log-space formulation make the system both pedagogically transparent and practically useful.

Possible extensions include:

- n-gram features or character-level models to reduce Spam/Scam confusion;
- online learning / incremental updates;
- integration of a more sophisticated tokeniser (stemming, lemmatisation);
- calibration of the posterior probabilities for cost-sensitive decision thresholds.

Acknowledgments

This work was carried out as part of the C++ course at the International Institute of Information Technology, Bangalore.

References

- [1] C. D. Manning, P. Raghavan and H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] K. P. Murphy. *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.